Loops Questions

1. Write a loop that prints all numbers from 1 to 10. (Easy)

   Explanation: Initialise a loop variable at 1 and iterate up to 10, printing each value. This demonstrates a basic counting loop.

2. Print all even numbers between 1 and N (inclusive), given N. (Easy)

   Explanation: Loop from 1 to N and print the number if it is divisible by 2. This uses a loop and an if condition to filter even numbers.

3. Print all odd numbers between 1 and N (inclusive), given N. (Easy)

   Explanation: Loop from 1 to N and print the number if it is not divisible by 2. This uses a loop and an if condition to filter odd numbers.

4. Calculate and print the sum of all numbers from 1 to N. (Easy)

   Explanation: Initialise a sum at 0 and loop through

numbers 1 to N, adding each number to the sum. After the loop, output the total sum.

5. Calculate the product of all numbers from 1 to N. (Medium)

   Explanation: Initialise a product variable at 1 and loop from 1 to N, multiplying each number to the product. This computes the factorial of N without using recursion.

6. Print the Fibonacci sequence up to N terms. (Medium)

   Explanation: Initialise variables to store the first two Fibonacci numbers and use a loop to compute each next number by summing the previous two. Output each term in sequence.

7. Display the multiplication table (from 1 to 10) for a given number X. (Medium)

   Explanation: Use a loop from 1 to 10 and multiply the loop index by X, printing the result at each iteration. This produces the multiplication table for X.

8. Count down from N to 1 and print each number. (Easy)

   Explanation: Start a loop variable at N and decrement it down to 1, printing each value. This shows how to loop in reverse order.

9. Use a while loop to compute the factorial of N. (Medium)

   Explanation: Initialise a result to 1 and loop while decrementing N (or incrementing a counter up to N), multiplying each step into the result. This calculates N! iteratively.

10. Use a do...while loop to print numbers from 1 to 5. (Easy)

    Explanation: Use a do...while loop with a counter starting at 1, logging the number then incrementing, and repeat until the counter exceeds 5.

11. Use a break statement to exit a loop early (e.g., stop printing numbers at 5). (Easy)

    Explanation: Loop from 1 upwards but include a condition if (i === 5) break; to exit the loop when 5 is reached, thus stopping early.

12. Use a continue statement to skip printing even numbers when looping from 1 to 10. (Easy)

    Explanation: Loop from 1 to 10 and use if (i % 2 === 0) continue; to skip even numbers; the loop continues with the next iteration.

13. Generate an array of the first N square numbers using a loop. (Easy)

    Explanation: Initialise an empty array and use a loop from 1 to N, computing i * i at each step and pushing it to the array. This creates a list of squares.

14. Check if a number is prime using a loop. (Medium)

    Explanation: Loop from 2 up to the square root of the number, checking if any value divides it evenly. If so, the number is not prime; otherwise it is prime.

15. Find the largest number in an array using a loop. (Medium)

    Explanation: Initialise a variable to the first array element,

then iterate through the array comparing each element; update the variable if a larger element is found.

16. Reverse an array using a loop (create a new array with elements in reverse order). (Medium)

    Explanation: Use a loop from the last index to the first, pushing each element into a new array. This constructs a reversed copy of the original array.

17. Merge two arrays into a new array using loops (without using built-in methods like concat). (Medium)

    Explanation: Loop through the first array and push each element into a new array, then loop through the second array and do the same. This merges both arrays manually.

18. Find the average of numbers in an array using loops. (Easy)

    Explanation: Use a loop to sum all elements in the array and then divide by the array's length. This computes the average.

19. Collect all even numbers from an array into a new array using a loop. (Easy)

    Explanation: Initialise an empty array and loop through the original array, pushing each even element (those divisible by 2) into the new array.

20. Print a pattern of stars (e.g., right triangle) using nested loops. (Medium)

    Explanation: Use a loop for rows and a nested loop for columns to print a star pattern, increasing the number of stars on each line to form a triangle.

21. Use nested loops to find all pairs of numbers (one from each array) that sum to a given value. (Medium)

    Explanation: Nested loops iterate through each element of both arrays; for each pair, check if their sum equals the target and store or print those pairs.

22. Iterate over an object's properties using a for...in loop and print key-value pairs. (Medium)

    Explanation: Use for (let key in obj) to loop through each

property key, then access the corresponding value as obj[key], printing both.

23. Use a for...of loop to sum the elements in an array. (Easy)

    Explanation: Initialise a sum to 0 and use for (let num of arr) to iterate through the array, adding each element to the sum.

24. Use a for...in loop to print the keys and values of an object. (Medium)

    Explanation: Similar to the above, use for...in over the object and inside the loop print key: obj[key] to display each property and its value.

25. Remove duplicates from an array using loops. (Medium)

    Explanation: Loop through the array and use another (or indexOf) check to keep track of seen elements; build a new array containing only the first occurrence of each value.

26. Flatten a 2D array into a 1D array using nested loops. (Medium)

    Explanation: Loop through each sub-array and inside that

loop through each element of the sub-array, pushing each element into a new flat array.

27. Filter out (remove) all odd numbers from an array using loops. (Easy)

    Explanation: Initialise a new array and loop through the original; push only the even numbers into the new array, effectively removing odd numbers

28. Rotate the elements of an array to the right by one position using a loop. (Medium)

    Explanation: Save the last element, then loop backwards through the array shifting each element to the next index, and finally put the saved last element at the first index.

29. Given an array of strings, loop through and print the length of each string. (Medium)

    Explanation: Use a loop (e.g., for or for...of) to go through each string in the array and log string.length to measure its length.

30. Count down from 10 to 1 using a loop. (Easy)

    Explanation: Initialise a loop variable at 10 and decrement down to 1, printing each number. This uses a reverse loop.

31. Create a multiplication table (1 through 10) using nested loops. (Medium)

    Explanation: Use an outer loop for multiplicand (1 to 10) and an inner loop for multiplier (1 to 10), printing their product for each combination.

32. Use nested loops to create an identity matrix of size N (2D array with 1s on the diagonal). (Medium)

    Explanation: Use two nested loops; in the inner loop, if row index equals column index put 1, otherwise 0. This produces an identity matrix.

33. Given an array of strings, loop through and print each string prefixed with 'Task:'. (Easy)

    Explanation: Use a loop over the array and for each element str output 'Task: ' + str to prefix each string with 'Task:'.

34. Create an array containing the first N multiples of a given number X using a loop. (Medium)

    Explanation: Initialise an empty array and loop from 1 to N, pushing i * X on each iteration to get multiples of X.

35. Simulate rolling a six-sided die N times using a loop and calculate the total sum. (Medium)

    Explanation: Loop N times and each time add a random integer between 1 and 6 to a sum variable. This uses loops and random number generation to simulate dice rolls.

36. Use a loop to sum only the positive numbers in an array, skipping negative values. (Easy)

    Explanation: Loop through the array and add each number to a sum only if it's positive (greater than or equal to 0). This uses conditional filtering inside a loop.

37. Repeat a string N times using a loop (without using the built-in repeat method). (Easy)

    Explanation: Initialise an empty result and loop N times,

appending the string in each iteration. This manually repeats the string.

38. Copy all properties from one object to another using a loop. (Medium)

    Explanation: Use a for...in loop over the source object's properties and assign each value to the target object, copying all key-value pairs.

39. Write a loop that prints 'Fizz' for multiples of 3 and 'Buzz' for multiples of 5 from 1 to 100. (Medium)

    Explanation: Loop from 1 to 100; for each number check if divisible by 3 or 5 and print 'Fizz', 'Buzz', or nothing accordingly. This shows use of conditionals in loops.

40. Given an array, use a loop to create a reversed copy without modifying the original array. (Medium)

    Explanation: This is similar to reversing, but emphasises leaving the original untouched: use a loop from end to start, pushing elements into a new array.

41. Sum all elements in a two-dimensional array using nested loops. (Medium)

    Explanation: Use a loop for each sub-array, and within that loop, iterate through its elements adding each to a running total. This handles multi-dimensional arrays.

42. Given an array of objects, use a loop to print a specific property of each object (e.g., 'name'). (Medium)

    Explanation: Use a loop (for, for...of, etc.) to iterate through the array, then access and print the desired property of each object inside the loop.

43. Check if a number exists in an array (linear search) using a loop. (Easy)

    Explanation: Loop through the array and compare each element to the target number. If a match is found, break or note that the number exists.

44. Calculate the product of all positive numbers in an array using a loop. (Medium)

    Explanation: Initialize a product variable to 1. Loop

through the array and if an element is positive, multiply it to the product. This computes the product only of positives.

45. Given a 2D array (matrix), use loops to find the sum of each row. (Medium)

    Explanation: Use an outer loop to iterate each sub-array (row), then an inner loop to add up its elements. Print or store the sum for each row.

46. Determine if a given number is a power of two using a loop. (Medium)

    Explanation: Start from 1 and repeatedly multiply by 2 until it is equal to or greater than the number. If it becomes equal at some point, the number is a power of two.

47. Swap the first and last elements of an array using a loop. (Easy)

    Explanation: Use a temporary variable to hold the first

element, assign the last element to the first index, and then assign the saved first element to the last index.

48. Given an array, compute the running sum (cumulative sum) using a loop. (Easy)

    Explanation: Initialise a running sum to 0 and loop through the array, adding each element to the running sum and storing it back in the array (or a new array).

49. Find the most frequent element in an array using loops. (Medium)

    Explanation: Use an object or map to count each element's occurrences by looping through the array. Then determine which element has the highest count.

50. Concatenate all elements of an array into a single string using a loop. (Easy)

    Explanation: Initialise an empty string and loop through the array, appending each element (converted to string) to it. This joins all array elements into one string.

Functions Questions

1. Write a function that takes two numbers and returns their sum. (Easy)

   Explanation: Define a function with two parameters and use the + operator to return their sum.

2. Write a function that multiplies two numbers. (Easy)

   Explanation: Define a function with two parameters and return the product using the * operator.

3. Write a function to convert temperature from Celsius to Fahrenheit. (Easy)

   Explanation: Use the formula F = C * (9/5) + 32 inside the function and return the result.

4. Write a function that returns the square of a number. (Easy)

   Explanation: Define a function with one parameter and return that parameter multiplied by itself.

5. Write a function that returns the larger of two numbers. (Easy)

   Explanation: Use an if statement to compare the two parameters and return the larger value.

6. Write a function that checks if a number is even or odd. (Easy)

   Explanation: Check the number modulo 2; if number % 2 === 0, it's even, else it's odd, and return the result.

7. Write a function that checks whether a string is a palindrome. (Medium)

   Explanation: Compare the string to its reverse

(which can be obtained with a loop or built-in method) and return true if they match.

8. Write a recursive function to calculate the factorial of N. (Medium)

   Explanation: If N is 0 or 1 return 1; else return N * factorial(N-1). This uses recursion to solve the problem.

9. Write a recursive function to compute the Nth Fibonacci number. (Medium)

   Explanation: If N is 0 or 1, return N; otherwise return fib(N-1) + fib(N-2). This recursion follows the Fibonacci definition.

10. Write a function that takes any number of arguments and returns their sum. (Medium)

    Explanation: Use the rest parameter syntax (...args)

to gather all arguments into an array, then loop through the array summing all values.

11. Write an arrow function that doubles a number. (Easy)

    Explanation: Use the arrow syntax: const double = (x) => x * 2;. This returns twice the input.

12. Write a function that finds the smallest element in an array. (Easy)

    Explanation: Loop through the array keeping track of the smallest value seen so far, and return that value after the loop.

13. Write a function that reverses a string. (Easy)

    Explanation: Split the string into characters or use a loop from end to start to build a reversed string, then return it.

14. Write a function to count the number of vowels in a string. (Medium)

    Explanation: Loop through each character of the string and increment a count for each character that is a vowel (e.g., aeiou).

15. Write a function that returns the largest element in an array. (Easy)

    Explanation: Loop through the array, tracking the maximum value found. Return the largest value after the loop.

16. Write a function that merges two objects into one (shallow merge). (Medium)

    Explanation: Create a new object and copy all properties from both input objects into it (e.g., using a loop or Object.assign).

17. Write an immediately-invoked function expression (IIFE) that logs "Hello, World!". (Easy)

    Explanation: Define a function inside parentheses and immediately call it to execute the code that logs the greeting.

18. Write a function that returns another function which adds a given number to its argument (closure). (Medium)

    Explanation: Create a function that takes x and returns a new function y => x + y. The inner function adds the original x to its argument.

19. Write a function that mimics the behavior of Array.map (takes an array and a callback, returns a new array). (Medium)

    Explanation: Define a function that loops over each element of the input array, applies the callback to it,

and pushes the result into a new array, which is returned.

20. Write a function that mimics the behavior of Array.filter (takes an array and a predicate, returns a new array). (Medium)

    Explanation: Define a function that loops through the array and pushes elements that satisfy the predicate into a new array, then return it.

21. Write a function to capitalize the first letter of a string. (Easy)

    Explanation: Take the first character (str[0]), convert it to uppercase, and concatenate it with the rest of the string (starting from index 1).

22. Write a function to check if a year is a leap year. (Easy)

    Explanation: Return true if the year is divisible by 4

but not by 100, or if it is divisible by 400. This implements the leap year rules.

23. Write a function that returns the type of a variable (e.g., 'string', 'number'). (Easy)

    Explanation: Use the typeof operator on the parameter and return the result.

24. Write a function that returns a random integer between min and max. (Easy)

    Explanation: Generate a random number using Math.random(), scale it to the range [min, max], and use Math.floor to get an integer.

25. Write a function that checks if a number is prime. (Medium)

    Explanation: Inside the function, loop from 2 up to the square root of the number to test divisibility. Return false if any divisor is found, otherwise true.

26. Write a function that removes duplicate items from an array. (Medium)

    Explanation: Loop through the array and track seen values (in an object or new array); build and return a new array without the duplicates.

27. Write a function that rotates an array to the right by one position. (Easy)

    Explanation: Use pop() to remove the last element and unshift() to add it to the front of the array, then return the modified array.

28. Write a function to sum the digits of a number. (Easy)

    Explanation: Convert the number to a string or repeatedly modulo and divide by 10 to extract digits, summing them as you go.

29. Write a function that flattens an array of depth 1 (one level of nesting). (Medium)

    Explanation: Loop through the outer array and for each sub-array, loop through its elements and push them to a new result array, effectively flattening one level.

30. Write a function that returns the longest string in an array of strings. (Easy)

    Explanation: Loop through the array tracking the longest string seen so far (by length), and return it after checking all.

31. Write a function that checks if an array is sorted in ascending order. (Medium)

    Explanation: Loop through the array and compare each element to the previous one; if you find any

element smaller than its predecessor, return false; otherwise, true.

32. Write a function that counts the occurrences of a character in a string. (Easy)

    Explanation: Loop through the string and increment a count each time the target character is encountered.

33. Write a function that returns the median of three numbers. (Medium)

    Explanation: Use conditional statements or sort the three numbers to find the middle value (the one that is neither max nor min).

34. Write a function that swaps keys and values of an object (new object with values as keys). (Medium)

    Explanation: Loop through the object properties

and create a new object where newObj[value] = key for each key: value.

35. Write a function that adds a prefix to all strings in an array. (Easy)

    Explanation: Loop through the array and for each string, concatenate the prefix before it, storing or logging the result.

36. Write a function that generates an array of N random integers between 1 and 100. (Easy)

    Explanation: Initialize an empty array and loop N times, each time pushing Math.floor(Math.random()*100) + 1 into the array.

37. Write a function that checks if two objects have the same keys and values (shallow comparison). (Medium)

    Explanation: First check if they have the same

number of keys, then loop through the keys of one object and verify each key-value matches in the other.

38. Write a function that memoizes (caches) the results of another function (e.g., sum of two numbers). (Medium)

    Explanation: Return a function that keeps a cache object; if the function is called with known arguments, return cached result; otherwise compute, store in cache, and return.

39. Write a function that prints each element of an array with its index. (Easy)

    Explanation: Loop through the array (using index) and print a message like Index i: element for each item.

40. Write a function that delays execution of a callback by N milliseconds (simulating debounce). (Medium)

    Explanation: Use setTimeout inside the function to call the provided callback after N milliseconds.

41. Write a function that performs a shallow clone of an object. (Medium)

    Explanation: Create a new object and copy all enumerable properties from the original (e.g., using a loop or Object.assign).

42. Write a function that merges two sorted arrays into one sorted array (merge step of merge sort). (Medium)

    Explanation: Use two pointers to traverse both arrays, comparing elements and pushing the smaller into a result array until all elements are merged.

43. Write a function that returns the first non-repeated character in a string. (Medium)

    Explanation: Loop through the string and use a map/object to count occurrences. Then loop again to find the first character with count 1.

44. Write a function that converts a number to its binary string representation. (Medium)

    Explanation: You can use built-in toString(2) or loop by dividing the number by 2 repeatedly and constructing the binary digits.

45. Write a function that returns the number of digits in an integer. (Easy)

    Explanation: Convert the number to a string and return length, or loop dividing by 10 until the number is 0, counting iterations.

46. Write a function to find the greatest common divisor (GCD) of two numbers. (Medium)

    Explanation: Use the Euclidean algorithm: loop while the second number is not zero, swapping and taking remainders, until the GCD is found.

47. Write a function that converts a kebab-case or snake_case string into camelCase. (Medium)

    Explanation: Split the string by '-' or '_', capitalize the first letter of subsequent words, and join them without separators.

48. Write a function that prints all even numbers from an array. (Easy)

    Explanation: Loop through the array and print each element that is divisible by 2. This filters the array for evens.

49. Write a function that returns the sum of all elements in an array. (Easy)

    Explanation: Loop through the array, adding each element to a running total, and return the total after the loop.

50. Write a function that checks if two strings are anagrams of each other. (Medium)

    Explanation: Sort both strings or use counting; then compare if the sorted strings are equal or counts match.

51. Write a function that returns the intersection of two arrays. (Medium)

    Explanation: Loop through one array and check if each element exists in the second (e.g., via includes). Collect common elements in a new array.

Arrays Questions

1. Write a function to create an array with numbers from 1 to N. (Easy)

   Explanation: Use a loop to push numbers from 1 up to N into an array, then return that array.

2. Given an array, write code to find its length without using the length property. (Medium)

   Explanation: Loop through the array until you reach

an undefined index or catch an error, counting iterations; this mimics finding the length.

3. Write a function to get the first element of an array. (Easy)

   Explanation: Return array[0], which is the first element (assuming the array is non-empty).

4. Write a function to get the last element of an array. (Easy)

   Explanation: Return array[array.length - 1], which accesses the last element of the array.

5. Write a function to add an element to the end of an array. (Easy)

   Explanation: Use push(element) on the array, which appends the element to the end.

6. Write a function to add an element to the beginning of an array. (Easy)

Explanation: Use unshift(element) on the array, which inserts the element at the start.

7. Write a function to remove the last element of an array. (Easy)

   Explanation: Use pop() on the array, which removes and returns the last element.

8. Write a function to remove the first element of an array. (Easy)

   Explanation: Use shift() on the array, which removes and returns the first element.

9. Write a function to copy an array (shallow copy). (Easy)

   Explanation: Use slice() with no arguments or spread syntax ([...]) to create a shallow copy of the array.

10. Write a function to check if an array contains a given value. (Easy)

    Explanation: Loop through the array and compare each element to the target value, returning true if found.

11. Write a function to reverse an array in place. (Medium)

    Explanation: Swap elements from the ends towards the middle using a loop; or use the built-in reverse() method.

12. Write a function to sort an array of numbers in ascending order without using the sort method. (Medium)

    Explanation: Implement a simple sorting algorithm (like bubble sort): loop through the array repeatedly and swap adjacent elements if out of order.

13. Write a function to find the largest number in an array. (Easy)

    Explanation: Loop through the array tracking the maximum value found and return it.

14. Write a function to find the smallest number in an array. (Easy)

    Explanation: Loop through the array tracking the minimum value found and return it.

15. Write a function to find the sum of all elements in an array. (Easy)

    Explanation: Loop through the array adding each element to a running total, and return the total.

16. Write a function to compute the average of numbers in an array. (Easy)

    Explanation: Sum the elements (using a loop) and divide by the array length.

17. Write a function that returns a new array with all elements multiplied by 2. (Easy)

    Explanation: Loop through the array and push element * 2 into a new array, which is then returned.

18. Write a function that filters out (removes) all even numbers from an array. (Medium)

    Explanation: Loop through the array and push only the odd numbers into a new array, effectively filtering out evens.

19. Write a function that returns the index of a given element in an array (or -1 if not found). (Easy)

    Explanation: Loop through the array and compare each element to the target; return the index when found, otherwise -1.

20. Write a function to remove an element by value from an array (first occurrence). (Medium)

Explanation: Loop through the array to find the element, then use splice at that index to remove it.

21. Write a function that concatenates two arrays into a new array. (Easy)

    Explanation: Loop through each array and push its elements into a result array (or use spread operator [...]).

22. Write a function to flatten an array of arrays by one level. (Medium)

    Explanation: Loop through the outer array; if an element is itself an array, loop through it and push its elements to a new array.

23. Write a function to find the maximum sum of any contiguous subarray (Kadane's algorithm). (Medium)

    Explanation: Use a loop while tracking the current

running sum and maximum sum found so far (reset running sum to 0 if it becomes negative).

24. Write a function that finds the intersection (common elements) of two arrays. (Medium)

    Explanation: Loop through one array and use includes on the other to collect common elements.

25. Write a function that finds the union of two arrays (all unique elements). (Medium)

    Explanation: Combine both arrays and use a loop to build a new array, adding an element only if it isn't already included.

26. Write a function that checks if an array is sorted in ascending order. (Easy)

    Explanation: Loop through the array checking if any element is greater than the next; if so, return false; else true.

27. Write a function that removes duplicates from an array and returns the new array. (Medium)

    Explanation: Loop through the array, and for each element, use a set or new array to track unique values, only add unseen elements.

28. Write a function that merges two sorted arrays into a single sorted array. (Medium)

    Explanation: Use two pointers to traverse both sorted arrays, always picking the smaller current element to add to the result array.

29. Write a function to count how many times a given number appears in an array. (Easy)

    Explanation: Loop through the array and increment a counter each time the target number is encountered.

30. Write a function to split an array into two halves. (Easy)

    Explanation: Use slice(0, mid) and slice(mid) to divide the array, where mid is half the array's length.

31. Write a function to find the first unique (non-repeating) element in an array. (Medium)

    Explanation: Loop through the array and count occurrences of each element (using an object), then loop again to find the first with count 1.

32. Write a function to shuffle an array (randomize order of elements). (Medium)

    Explanation: Implement the Fisher-Yates shuffle algorithm: loop backwards and swap each element with a random earlier element.

33. Write a function to rotate an array to the left by k positions. (Medium)

    Explanation: Use splice and push or loops: remove the first element and append it at the end, repeat this k times.

34. Write a function to chunk an array into smaller arrays of size N. (Medium)

    Explanation: Loop through the array in steps of N, using slice to extract chunks of length N and pushing them into a result array.

35. Write a function to find the second largest number in an array. (Medium)

    Explanation: Loop through the array to track the largest and second-largest values, updating them as you find larger elements.

36. Write a function to get all indices of a given value in an array. (Medium)

    Explanation: Loop through the array and collect indexes where the element equals the target value.

37. Write a function to sum the diagonal elements of a square matrix (2D array). (Medium)

    Explanation: Loop index i from 0 to n-1 and add matrix[i][i] to a sum (for main diagonal).

38. Write a function to transpose a square matrix (swap rows and columns). (Medium)

    Explanation: Use nested loops to swap matrix[i][j] with matrix[j][i] for all i, j to transpose the matrix.

39. Write a function to remove all instances of a value from an array. (Medium)

    Explanation: Loop through the array and build a

new array containing only those elements that are not equal to the given value.

40. Write a function to split an array into two parts at a given index. (Easy)

    Explanation: Use slice(0, index) and slice(index) to split the array at the given index.

41. Write a function that implements binary search on a sorted array. (Medium)

    Explanation: Use a loop (or recursion) to repeatedly divide the search interval by two, comparing the target to the middle element.

42. Write a function to rotate an array to the right by k positions. (Medium)

    Explanation: Use splice and unshift or loops: remove the last element and insert it at the front, repeating k times.

43. Write a function that returns the prime numbers in an array. (Medium)

    Explanation: Loop through the array and for each number check primality (as in earlier), collecting primes into a result array.

44. Write a function that converts an array of numbers to their string representations. (Easy)

    Explanation: Loop through the array and use toString() or string concatenation to convert each number to a string.

45. Write a function that creates an array of arrays representing a 2D matrix of zeros (size MxN). (Medium)

    Explanation: Use nested loops: first loop for rows, inside loop for columns, pushing 0 into an inner array, then that inner array into the matrix.

46. Write a function to delete all null or undefined values from an array. (Medium)

    Explanation: Loop through the array and push only non-null, non-undefined elements into a new array.

47. Write a function that checks if two arrays have any elements in common. (Medium)

    Explanation: Use a loop: for each element in the first array, check if it exists in the second (e.g., includes). Return true if any match.

48. Write a function to create a copy of an array without its first and last elements. (Easy)

    Explanation: Use slice(1, -1) to create a new array excluding the first and last elements.

49. Write a function that returns a random element from an array. (Easy)

    Explanation: Use Math.random() to pick a random

index (Math.floor(Math.random() * array.length)) and return the element at that index.

50. Write a function that calculates the dot product of two equal-length arrays of numbers. (Medium)

    Explanation: Loop through both arrays, multiply corresponding elements and sum the results. Return the total.

Strings Questions

1. Write a function that returns the length of a string. (Easy)

   Explanation: Use the length property (e.g., str.length) to get the number of characters.

2. Write a function that concatenates two strings. (Easy)

   Explanation: Use the + operator or .concat() method to join the two strings.

3. Write a function to convert a string to uppercase. (Easy)

   Explanation: Use the toUpperCase() method on the string to convert all characters to uppercase.

4. Write a function to convert a string to lowercase. (Easy)

   Explanation: Use the toLowerCase() method on the string to convert all characters to lowercase.

5. Write a function that trims whitespace from the start and end of a string. (Easy)

   Explanation: Use the trim() method to remove leading and trailing whitespace.

6. Write a function that splits a string by spaces into an array of words. (Easy)

   Explanation: Use the split(' ') method to divide the string at each space, producing an array of words.

7. Write a function to check if a string starts with a given substring. (Easy)

   Explanation: Use the startsWith(substring) method

to return true if the string begins with that substring.

8. Write a function to check if a string ends with a given substring. (Easy)

   Explanation: Use the endsWith(substring) method to return true if the string ends with that substring.

9. Write a function to check if a string contains a given substring. (Easy)

   Explanation: Use the includes(substring) method or indexOf to test if the substring is found.

10. Write a function that returns the character at a given index in a string. (Easy)

    Explanation: Use bracket notation str[index] or charAt(index) to get the character.

11. Write a function to replace all occurrences of a substring in a string. (Medium)

Explanation: Use split and join (or regex with replaceAll) to replace all instances of the substring.

12. Write a function that reverses a string. (Medium)

    Explanation: Split the string into an array of characters, reverse the array, and join it back into a string.

13. Write a function that checks if a string is a palindrome. (Medium)

    Explanation: Compare the string to its reverse (use the reversing method above) and return true if they are the same.

14. Write a function that converts a string to Title Case (capitalizing first letter of each word). (Medium)

    Explanation: Split the string into words, capitalize the first letter of each word and lowercase the rest, then join back.

15. Write a function that counts the number of vowels in a string. (Medium)

    Explanation: Loop through the string and count characters that match vowels (a, e, i, o, u). Return the count.

16. Write a function that removes all vowels from a string. (Medium)

    Explanation: Loop through or use regex to filter out vowels and rebuild the string without them.

17. Write a function to check if two strings are anagrams of each other. (Medium)

    Explanation: Sort the characters of both strings and compare, or count character frequencies and compare.

18. Write a function to convert a string into an array of characters. (Easy)

Explanation: Use split('') to split the string into individual characters.

19. Write a function that capitalizes the first letter of each word in a string. (Medium)

    Explanation: Similar to Title Case: split by space, capitalize first letter of each word, join back.

20. Write a function to count the occurrences of a character in a string. (Easy)

    Explanation: Loop through the string and increment a counter each time you see the target character.

21. Write a function that removes duplicate characters from a string. (Medium)

    Explanation: Loop through the string and use a new string or set to build only unique characters.

22. Write a function to check if a string contains only digits. (Easy)

Explanation: Use a loop or regex to verify each character is between '0' and '9'.

23. Write a function that returns the index of the first non-repeated character in a string. (Medium)

    Explanation: Use an object to count character occurrences, then loop again to find the first with count 1.

24. Write a function that checks if one string is a rotation of another. (Medium)

    Explanation: Concatenate one string with itself and use includes to see if the other string is a substring.

25. Write a function that converts numbers written in English words to digits (e.g., 'four' to 4). (Medium)

    Explanation: Create a mapping of words to numbers and use it to transform the input word to the corresponding digit.

26. Write a function that pads a number with leading zeros to make it N digits long. (Easy)

    Explanation: Convert the number to a string and use padStart(N, '0') or manually prepend zeros until the length is N.

27. Write a function that finds and returns all indices of a substring in a string. (Medium)

    Explanation: Use a loop with indexOf(substring, startIndex) repeatedly to find all occurrences.

28. Write a function that checks if a string is empty or whitespace only. (Easy)

    Explanation: Trim the string and check if the length is zero.

29. Write a function that swaps the first and last character of a string. (Easy)

    Explanation: Construct a new string with the last

character at the front, the middle unchanged, and the first character at the end.

30. Write a function that counts words in a string. (Medium)

    Explanation: Split the string by spaces (and possibly filter out empty entries) and count the resulting words.

31. Write a function that censors a list of bad words in a string by replacing them with '***'. (Medium)

    Explanation: Loop through the list of bad words and use replace or regex to replace each occurrence with asterisks.

32. Write a function that trims all leading zeros from a numeric string. (Medium)

    Explanation: Use a loop or regex to remove '0' from

the start of the string until a non-zero digit is reached.

33. Write a function that formats a number with commas as thousands separators. (Medium)

    Explanation: Convert to string and use regex or loop to insert commas every three digits from the right.

34. Write a function that removes all non-alphanumeric characters from a string. (Medium)

    Explanation: Use a regex like /[^a-z0-9]/gi to remove anything that isn't a letter or digit.

35. Write a function that converts a string of digits into an integer without using parseInt. (Medium)

    Explanation: Loop through each character, multiply the result by 10 and add the numeric value of the digit (using char code).

36. Write a function that finds the longest word in a sentence. (Medium)

    Explanation: Split the sentence into words and find the word with the maximum length.

37. Write a function that replaces spaces in a string with hyphens. (Easy)

    Explanation: Use split(' ') and join('-') to replace spaces with hyphens.

38. Write a function to compare two strings alphabetically. (Medium)

    Explanation: Use the < and > operators or localeCompare to compare strings.

39. Write a function that removes the last character from a string. (Easy)

    Explanation: Use slice(0, -1) or similar to drop the last character and return the new string.

40. Write a function that repeats each character in a string N times. (Medium)

    Explanation: Loop through each character and append it N times to a new string.

41. Write a function that checks if a string is a valid palindrome (ignore non-letter characters and case). (Medium)

    Explanation: Clean the string by removing non-letters and changing to the same case, then check palindrome by comparing to reverse.

42. Write a function that abbreviates a name (e.g., 'John Smith' to 'J. Smith'). (Medium)

    Explanation: Split the name, take first letter of the first name with a dot, and combine with the full last name.

43. Write a function to compute the Levenshtein distance between two strings. (Medium)

    Explanation: Use dynamic programming or recursion to compute the minimum edit distance. (This is a more advanced challenge.)

44. Write a function to split a camelCase string into separate words. (Medium)

    Explanation: Loop through the string and whenever an uppercase letter is found, treat it as the start of a new word (or use regex).

45. Write a function that removes all HTML tags from a string. (Medium)

    Explanation: Use a regular expression (e.g., /<[^>]*>/g) to replace tags with an empty string, leaving only text content.

46. Write a function to translate a sentence into Pig Latin. (Medium)

    Explanation: Split the sentence into words, then for each word move the first letter to the end and add 'ay'; join the words back into a sentence.

47. Write a function to count uppercase and lowercase letters in a string. (Medium)

    Explanation: Loop through the string and use a regular expression or character comparison to check if each character is upper or lower case, incrementing separate counters.

48. Write a function that checks if a string has all unique characters. (Medium)

    Explanation: Use a set or an object to track seen characters while looping. If any character repeats, return false; otherwise return true.

49. Write a function that compresses a string using run-length encoding. (Medium)

    Explanation: Loop through the string, counting consecutive identical characters, and build a compressed result like 'a3b2' for 'aaabb'.

50. Write a function that censors a bad word (replace it entirely with '*'). (Medium)

    Explanation: Find the bad word using indexOf and replace all its characters with * (for example using replace).

51. Write a function that converts an IP address string into a 32-bit integer. (Medium)

    Explanation: Split the IP by dots, shift and sum each octet ((o1<<24)+(o2<<16)+(o3<<8)+o4).

52. Write a function that reverses each word in a sentence (keeping word order). (Medium)

Explanation: Split the sentence into words, reverse each word (as above), then join them with spaces to reconstruct the sentence.

53. Write a function that finds the longest common prefix between an array of strings. (Medium)

    Explanation: Compare characters of the first string to all others until a mismatch; the common prefix is the matched portion.

DOM Manipulation Questions

1. Select an element by its ID and change its text content. (Easy)

   Explanation: Use document.getElementById('elementId') to get the element, then set its textContent property.

2. Select elements by their class name and apply a style change. (Easy)

   Explanation: Use document.getElementsByClassName('className')

to get a collection, loop through it and change style properties.

3. Create a new element and append it to the document body. (Easy)

   Explanation: Use document.createElement('div') to create it and document.body.appendChild() to add it to the page.

4. Remove an element with a given ID from the DOM. (Easy)

   Explanation: Select the element by ID and then use element.remove() or parent.removeChild(element) to remove it.

5. Change the background color of all <p> elements on the page. (Easy)

   Explanation: Use document.querySelectorAll('p') to

select all paragraphs and loop through them, setting style.backgroundColor.

6. Add a click event listener to a button that alerts a message. (Easy)

   Explanation: Use button.addEventListener('click', () => alert('message')) to attach the click handler.

7. Toggle the visibility of an element when a button is clicked. (Medium)

   Explanation: In the click handler, change the element’s style.display between 'none' and 'block' to show/hide it.

8. Create a list of items in HTML using JavaScript. (Medium)

   Explanation: Create a <ul> element, loop through an array of items, create <li> elements for each, set

their text, and append to the <ul>. Then add the <ul> to the DOM.

9. Change the text of all elements of a given class. (Medium)

   Explanation: Use document.getElementsByClassName or querySelectorAll to get the elements and loop through, setting textContent for each.

10. Append a new item to an existing <ul> list. (Easy)

    Explanation: Use document.createElement('li') to make a new list item, set its text, and use parentUl.appendChild(newLi).

11. Replace an existing DOM element with another element. (Medium)

    Explanation: Use parent.replaceChild(newElement,

oldElement) or oldElement.replaceWith(newElement) to replace it.

12. Change the src attribute of an <img> element. (Easy)

    Explanation: Select the <img> element and set its src property to a new URL.

13. Add a class to an element dynamically. (Easy)

    Explanation: Use element.classList.add('newClass') to add a class to the element.

14. Remove a class from an element dynamically. (Easy)

    Explanation: Use element.classList.remove('className') to remove that class.

15. Toggle a class on an element when it is clicked. (Medium)

    Explanation: In a click event handler, use element.classList.toggle('className') to add/remove the class.

16. Insert an element before another element. (Medium)

    Explanation: Use parentNode.insertBefore(newElement, referenceElement) to insert the new element before the reference element.

17. Find the first element with a given tag name and log its text. (Easy)

    Explanation: Use document.getElementsByTagName('tag')[0] to get the first element, then use textContent or innerText.

18. Clone a DOM element and append the clone to the body. (Medium)

    Explanation: Use element.cloneNode(true) to make a deep copy, then append the clone (e.g., document.body.appendChild(clone)).

19. Set multiple CSS styles on an element in one line. (Easy)

    Explanation: Set the style.cssText property or assign multiple style properties, e.g. element.style.cssText = 'color:red; background:blue;'.

20. Animate an element’s opacity from 0 to 1 using JavaScript. (Medium)

    Explanation: Use a loop (or setInterval) to gradually increase element.style.opacity from 0 to 1.

21. Write code to get the value of a data attribute (e.g., data-id) from an element. (Easy)

Explanation: Use element.getAttribute('data-id') or element.dataset.id to retrieve the value.

22. Add multiple list items to an existing list using innerHTML in one operation. (Easy)

    Explanation: Build an HTML string of <li> elements and set listElement.innerHTML to that string to add them all.

23. Change the placeholder text of an input field. (Easy)

    Explanation: Select the input element and set its placeholder property to a new string.

24. Remove all child elements from a given element. (Medium)

    Explanation: Use a loop (while the element has a firstChild) to remove element.firstChild repeatedly, or set innerHTML = ''.

25. Get the computed style (e.g., background color) of an element. (Medium)

    Explanation: Use getComputedStyle(element).getPropertyValue('background-color') to retrieve the computed style.

26. Append a new row to an existing HTML table. (Medium)

    Explanation: Use insertRow() and insertCell() or create elements to add a new <tr> and <td> to the table.

27. Remove a specific class from all elements on the page. (Medium)

    Explanation: Use querySelectorAll with that class, loop through the elements and remove the class from each.

28. Toggle dark mode by adding/removing a class on the <body> element. (Medium)

    Explanation: In an event (e.g., button click), use document.body.classList.toggle('dark-mode') to switch modes.

29. Append an image element with a given URL to a div. (Easy)

    Explanation: Create an <img> using document.createElement('img'), set its src, and append to the target div.

30. Set the innerHTML of an element to embed a new HTML structure. (Easy)

    Explanation: Use element.innerHTML = 'New HTML' to replace the element's content with HTML.

31. Get the number of child elements of an element. (Easy)

    Explanation: Use element.children.length or loop through childNodes and count element nodes.

32. Add an input field dynamically to a form when a button is clicked. (Medium)

    Explanation: In the button’s click handler, create a new <input> and use appendChild on the form.

33. Change the value of all inputs of type 'text' on the page. (Medium)

    Explanation: Use querySelectorAll('input[type=text]') to get them, then loop and set each value property.

34. Use event delegation to handle clicks on list items in a list. (Medium)

    Explanation: Add a click listener on the parent

and in the handler check event.target to respond if a <li> was clicked.

35. Replace the content of a <p> tag with the current date and time. (Easy)

    Explanation: Create a new Date object and set `paragraphElement.textContent = new Date().toString()`.

36. Add an image with alt text and caption to a gallery section. (Medium)

    Explanation: Create <img> and <figcaption> elements, set the src and alt on the image, text on caption, then append to the gallery container.

37. Remove a list item from a <ul> when it is clicked. (Medium)

    Explanation: Attach a click listener to each <li> or

use delegation; on click, call item.remove() to delete it.

38. Find the form named 'myForm' and submit it via JavaScript. (Medium)

    Explanation: Use document.forms['myForm'].submit() to trigger form submission.

39. Add a data-index attribute to each item in a list representing its position. (Medium)

    Explanation: Loop through list items, and set item.dataset.index = i for each index i.

40. Sort elements of a list alphabetically by their text content. (Medium)

    Explanation: Get all items, convert to array, sort by text (via localeCompare), then append them in sorted order.

41. Clone a form element (with all child inputs) and append it to the page. (Medium)

    Explanation: Use form.cloneNode(true) to make a deep copy, and appendChild to add the cloned form.

42. Enable a hidden element by removing its hidden attribute. (Easy)

    Explanation: Select the element and call element.removeAttribute('hidden') or element.hidden = false.

43. Disable a button after it has been clicked once. (Easy)

    Explanation: In the button's click handler, set button.disabled = true to disable it.

44. Get all elements that have a specific data attribute (e.g., data-role). (Easy)

Explanation: Use querySelectorAll('[data-role]') to select all elements with that data attribute.

45. Add a click event listener to multiple buttons at once. (Easy)

    Explanation: Use querySelectorAll to get all buttons and loop through, attaching addEventListener('click', ...) to each.

46. Remove the last child of a list (<ul> or <ol>). (Easy)

    Explanation: Use listElement.removeChild(listElement.lastElementChild) to remove the final list item.

47. Change the color of an element based on mouse hover (mouseover/mouseout events). (Medium)

    Explanation: Use addEventListener('mouseover', ...) and mouseout to change style.color or other styles on hover.

48. Insert an HTML element at a specific position within its parent. (Medium)

    Explanation: Use parent.insertBefore(newElement, referenceChild) where referenceChild is the element that will follow the inserted one.

49. Dynamically create a table (e.g., 3 rows, 4 columns) in the DOM. (Medium)

    Explanation: Use document.createElement('table'), then loop to add rows (<tr>) and cells (<td>) to make 3x4.

50. Update the text of a span element when an input value changes (input event). (Medium)

    Explanation: Add an input event listener on the text field and set the span's textContent to the input's value.

51. Get a list of all form input values and log them. (Medium)

    Explanation: Use document.querySelectorAll('form input'), loop over them, and log input.value for each.

Asynchronous JavaScript Questions

1. Write code using setTimeout to log "Hello" after 2 seconds. (Easy)

   Explanation: Use setTimeout(() => console.log('Hello'), 2000) to schedule the message after 2000 milliseconds.

2. Write code using setInterval to log "Ping" every 3 seconds. (Easy)

   Explanation: Use setInterval(() => console.log('Ping'), 3000) to repeatedly log every 3 seconds.

3. Write a function that takes a callback and calls it after a 1-second delay. (Medium)

   Explanation: Define a function that takes a callback, and inside it use setTimeout(() => callback(), 1000) to call the callback after 1 second.

4. Use XMLHttpRequest to make a GET request to a public API (e.g., JSONPlaceholder) and log the response status. (Medium)

   Explanation: Create an XMLHttpRequest, use open('GET', url), send(), and add a load event listener to log xhr.status.

5. Use the Fetch API to get data from a URL and process the JSON response. (Medium)

   Explanation: Use fetch(url).then(response => response.json()).then(data => { /* handle data */ }).

6. Write an async function that uses await to wait for the Fetch API call to complete and then logs the data. (Medium)

   Explanation: Use async function, then const response = await fetch(url) and const data = await response.json(), then log or return the data.

7. Write a function that returns a Promise which resolves with "Success" after 1 second. (Easy)

   Explanation: Return a new Promise that calls resolve('Success') inside a setTimeout of 1000 ms.

8. Write code that uses .then() to handle the resolution of the above Promise and log the result. (Easy)

   Explanation: Call the promise function and then use .then(result => console.log(result)) to log 'Success'.

9. Demonstrate error handling in Promises by writing a Promise that rejects and using .catch(). (Medium)

   Explanation: Return a Promise that calls reject('Error'), and after calling it use .catch(error => console.log(error)).

10. Write an async function with a try/catch block that awaits a rejected Promise. (Medium)

    Explanation: In the async function, await the promise inside a try block, and handle the error in the catch block.

11. Use Promise.all to run multiple fetch requests and process results together. (Medium)

Explanation: Call Promise.all([fetch(url1), fetch(url2)]), then in .then use Promise.all again on JSON conversions, handling both results.

12. Use Promise.race to get the result of the first of two Promises. (Medium)

    Explanation: Call Promise.race([promise1, promise2]) and attach .then(result => ...) to get whichever resolves first.

13. Write a function to wrap setTimeout in a Promise (to await a delay). (Medium)

    Explanation: Return a Promise inside the function that uses setTimeout to resolve after the specified delay.

14. Write code that handles a rejected fetch (e.g., wrong URL) using .catch(). (Medium)

    Explanation: Use

`fetch(wrongUrl).then(...).catch(error => console.log('Fetch failed:', error))`.

15. Create an async function that fetches data, waits 1 second (using a delay promise), and then returns combined results. (Medium)

    Explanation: Inside an async function, use await `fetch`, `then await response.json()`, `await delay(1000)`, and return or log the combined data.

16. Use addEventListener with an async callback (and await inside) for a button click event. (Medium)

    Explanation: Attach a 'click' listener to a button, and use an async function for the handler where you can await some async operations.

17. Write code to fetch data with XMLHttpRequest using Promises instead of callbacks. (Medium)

    Explanation: Wrap the XHR logic in a Promise

constructor and resolve in the load event, allowing you to use .then on it.

18. Use setTimeout to log numbers 1 to 5 with a 1-second delay between each, in sequence. (Medium)

    Explanation: Chain multiple setTimeout calls or use recursion/timeouts properly so that each number logs after the previous with a delay.

19. Demonstrate async/await by writing two asynchronous calls in sequence (first await one, then the other). (Medium)

    Explanation: In an async function, use await on first fetch or promise, then await on the second, processing results in order.

20. Use forEach to call an async function on each element of an array (and explain how to handle order of execution). (Medium)

Explanation: Note that forEach with async callbacks doesn't await; using a normal for loop or for...of is needed for sequential awaits.

21. Implement a timeout for a Promise: if it doesn't resolve within X ms, reject. (Medium)

    Explanation: Create a new promise that wraps the original one and uses setTimeout to reject after X ms if the original hasn't resolved.

22. Use async/await to handle fetching data and catching errors in a try/catch block. (Medium)

    Explanation: Inside an async function, use a try { const res = await fetch(url); } catch(e) { ... } to catch any fetch errors.

23. Simulate a sequence of animations using Promises to wait between steps. (Medium)

Explanation: Chain .then() calls or use await with a custom delay promise to perform steps with pauses.

24. Write code to periodically fetch data (polling) every 10 seconds. (Medium)

    Explanation: Use setInterval to call a fetch function every 10000 ms, handling the result each time.

25. Write a function that returns a promise that resolves only after multiple AJAX calls have completed. (Medium)

    Explanation: Use Promise.all on an array of fetch calls and return that promise.

26. Demonstrate the use of finally in a Promise chain. (Easy)

    Explanation: Create a promise, attach .then(...), .catch(...), and .finally(() => ...) to handle cleanup regardless of outcome.

27. Create a resolved Promise with a number and chain two .then() calls to double that number twice and log the result. (Easy)

    Explanation: Use Promise.resolve(1).then(x => x * 2).then(x => x * 2).then(console.log); this will log 4.

28. Write a function sleep(ms) that returns a Promise which resolves after ms milliseconds, and use it in an async function to pause execution. (Easy)

    Explanation: Inside sleep, return new Promise(resolve => setTimeout(resolve, ms)). Then in an async function, use await sleep(1000) to pause for 1 second.

29. Log 'First' to the console, then use setTimeout to log 'Second' after 1 second. (Easy)

    Explanation: Call console.log('First') and then

setTimeout(() => console.log('Second'), 1000). 'First' appears immediately, 'Second' after 1 second.

30. Write a function that returns a Promise resolving to the sum of two numbers after a 2-second delay. (Medium)

    Explanation: Return new Promise(resolve => setTimeout(() => resolve(a + b), 2000)). Use .then or await to get the result.

31. Use the Fetch API to send a POST request with JSON data and log the JSON response. (Medium)

    Explanation: Call fetch(url, {method: 'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify(data)}), then await response.json() to parse and log.

32. Make an AJAX POST request using XMLHttpRequest to send JSON data. (Medium)

Explanation: Use xhr.open('POST', url), set xhr.setRequestHeader('Content-Type','application/json'), xhr.send(JSON.stringify(data)), and handle xhr.onload.

33. Write code that fetches from an invalid URL and catches the network error with .catch(). (Medium)

    Explanation: Call fetch('invalid-url').catch(error => console.log('Fetch error', error)) to handle and log the error.

34. Use Promise.all with async/await to fetch two URLs in parallel and log both results. (Medium)

    Explanation: In an async function, do const [res1, res2] = await Promise.all([fetch(url1), fetch(url2)]), then parse each to JSON.

35. Write code to fetch JSON and use try/catch to handle a JSON parsing error if the response is not

valid JSON. (Medium)

Explanation: Inside an async function, try { const data = await response.json(); } catch (e) { console.log('JSON parse error'); }.

36. Use fetch to request an image URL and get a Blob from the response. (Medium)

    Explanation: Call fetch(imageUrl).then(res => res.blob()).then(blob => { /* use blob */ }); to process the image data.

37. Create a resolved Promise with a number and chain two .then() calls to double that number twice and log the result. (Easy)

    Explanation: Use Promise.resolve(1).then(x => x * 2).then(x => x * 2).then(console.log); this will log 4.

38. Wrap navigator.geolocation.getCurrentPosition in a Promise so it can be used with async/await.

(Medium)

Explanation: Return new Promise((resolve, reject) => navigator.geolocation.getCurrentPosition(resolve, reject)), then await this promise.

39. Write code to submit a form using fetch with method POST and FormData. (Medium)

    Explanation: Collect the form element, then use `fetch(url, { method: 'POST', body: new FormData(form) })`, and await the response.

40. Fetch data from an API and then call a cleanup function in .finally() no matter success or error. (Easy)

    Explanation: Use `fetch(url).then(...).catch(...).finally(() =>`

console.log('Cleanup done')) to always run the final block.

41. Use Promise.all to wait for multiple async operations, some of which may fail. (Medium)

    Explanation: Call Promise.allSettled([...]) or Promise.all([...]) with .catch() to see how multiple results are handled.

42. Implement a timeout for a fetch request using AbortController. (Advanced)

    Explanation: Create an AbortController, pass its signal to fetch, and use controller.abort() after a timeout. (This goes beyond beginner level.)

43. Demonstrate the event loop by logging 'start', then using setTimeout to log 'end' after 0ms. (Medium)

    Explanation: This shows that even 0ms delay waits until the current stack is done. For example,

`console.log('start'); setTimeout(() =>` console.log('end'), 0);.

44. Use async/await to fetch data from two APIs in sequence and combine the results. (Medium)

    Explanation: const res1 = await fetch(url1); const data1 = await res1.json(); const res2 = await fetch(url2); const data2 = await res2.json(); console.log(data1, data2);

45. Use callback functions to handle a button click and an AJAX request sequentially. (Medium)

    Explanation: For example, in the button's onclick, call a function that makes an AJAX request and processes the result.

46. Write a callback function for a setTimeout inside another setTimeout (callback hell example). (Medium)

Explanation: Nest setTimeout(() => { /* do something */ setTimeout(() => { /* do more */ }, 1000); }, 1000);.

47. Convert a callback-based function (like setTimeout) to return a Promise. (Medium)

    Explanation: Wrap the async call in a Promise: function delay(ms) { return new Promise(res => setTimeout(res, ms)); }.

48. Fetch data from an API and update the DOM with the result. (Medium)

    Explanation: Use fetch(url).then(r => r.json()).then(data => { document.getElementById('out').textContent = data.value; });.

49. Use async/await in an event handler (e.g., button click) to fetch data and log it. (Medium)

Explanation: button.onclick = async () => { const response = await fetch(url); const data = await response.json(); console.log(data); };.

50. Demonstrate using Promise.allSettled to handle a mix of successful and failed Promises. (Medium)

Explanation: Call Promise.allSettled(promisesArray).then(results => { console.log(results); }); to see which resolved or rejected.